

Selezione Dinamica della Dimensione del Campione

in Metodi di Ottimizzazione per il Machine Learning

La tesi presenta la selezione dinamica della dimensione del campione negli algoritmi di ottimizzazione per il machine learning su larga scala, fondandosi sul lavoro di Byrd, Chin, Nocedal e Wu (2012). L'idea centrale: invece di fissare a priori la dimensione del batch, si decide durante l'algoritmo se e quando aumentarla, guidandosi da stime statistiche della varianza del gradiente stocastico. Il lavoro offre un'analisi teorica completa della convergenza, il miglioramento di alcuni risultati mediante disuguaglianze più strette e un'implementazione Python dei quattro algoritmi principali: Dynamic GD, Newton-CG, estensione L_1 e BB-CCV, con un'applicazione web interattiva per la sperimentazione.

Struttura dell'esposizione

- Contesto: il costo del gradiente su larga scala e il mini-batch
- Stato dell'arte: SGD e metodi a varianza ridotta (SVRG, SAGA)
- L'idea: batch grandi solo quando servono, con la Condizione di Controllo della Varianza (CCV)
- I quattro algoritmi a campionamento dinamico e i loro schemi
- Risultati teorici di convergenza e complessità
- Esperimenti numerici: app web, problemi sintetici, riuso del mini-batch
- Conclusioni e lavoro futuro

Contesto: ottimizzazione su larga scala

- Obiettivo: minimizzare la perdita empirica

$$J(w) = \frac{1}{N} \sum_{i=1}^N \ell(w; i), \quad w \in \mathbb{R}^m$$

con $N \approx 10^6$ – 10^9 esempi e m grande: il gradiente completo $\nabla J(w)$ costa $O(Nm)$ per iterazione, proibitivo.

- Mini-batch $\mathcal{S}_k$ di dimensione n_k : stima $g_k = \nabla J_{\mathcal{S}_k}(w_k)$, costo $O(n_k m)$. Se $n_k \ll N$ le iterazioni sono economiche.
- Compromesso classico (Bottou–Bousquet, 2008): batch piccolo = stima rumorosa; batch grande = stima accurata ma costo proporzionale alla dimensione.
- Nei metodi classici n_k è **fissata a priori**; questa tesi analizza la proposta di renderla **dinamica** durante le iterazioni (Byrd, Chin, Nocedal e Wu, 2012).

Stato dell'arte: SGD e il problema della varianza

- SGD (Robbins–Monro, 1951): aggiornamento su un singolo esempio (o un piccolo batch), $w_{k+1} = w_k - \alpha_k \nabla \ell(w_k; i_k)$, con $\mathbb{E}[\nabla \ell(w_k; i)] = \nabla J(w_k)$: stimatore *non distorto*, costo per iterazione indipendente da N .
- Ma la varianza $\text{Var}[\nabla \ell(w; i)]$ **non** tende a zero vicino all'ottimo: quando $\nabla J(w) \rightarrow 0$ il rumore domina e limita la precisione raggiungibile.
- Conseguenze pratiche: a passo fisso si resta in un intorno dell'ottimo (precisione limitata); a passo $\alpha_k \rightarrow 0$ la convergenza rallenta.
- Su larga scala conta il **costo totale** (costo per iterazione $\times$ numero di iterazioni), non solo il numero di iterazioni.
- Due strade per uscirne: aumentare il campione (costo) oppure ridurre la varianza con stimatori migliori.

Metodi a varianza ridotta: l'idea comune

- SVRG (*Stochastic Variance Reduced Gradient*; Johnson–Zhang, 2013) e SAGA (metodo di riduzione della varianza di Defazio–Bach–Lacoste-Julien, 2014): SGD + *correzione* costruita su quantità globali.
- Forma generale dello stimatore corretto:

$$g_k = \nabla \ell(w_k; i_k) - c_k + \mathbb{E}[c_k],$$

che resta non distorto: $\mathbb{E}[g_k] = \nabla J(w_k)$.

- Se la correzione c_k è correlata al rumore del gradiente stocastico, la varianza di g_k si riduce (idealmente tende a zero vicino all'ottimo): *variance reduction*.
- Differenza chiave tra i due metodi:
 - **SVRG**: ricalcola periodicamente un gradiente esatto su un punto di ancoraggio;
 - **SAGA**: mantiene una tavola dei gradienti per-esempio, aggiornata a ogni iterazione.

Dalla tesi — Sez. 4.1 (Lavori Correlati) e Appendice D (Schemi dei metodi a varianza ridotta)

SVRG: l'ancora e il ricampionamento ogni m iterazioni

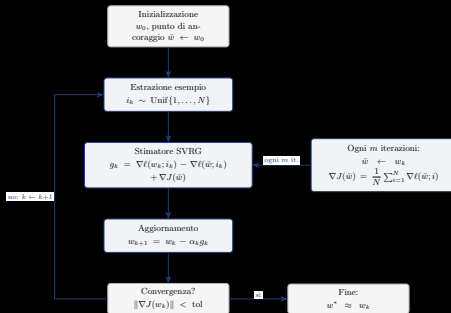
- Punto di ancoraggio $\bar{w}$ con gradiente esatto $\nabla J(\bar{w})$, ricalcolato ogni m iterazioni.

- Stimatore non distorto:

$$g_k = \nabla \ell(w_k; i_k) - \nabla \ell(\bar{w}; i_k) + \nabla J(\bar{w})$$

- **Idea:** finché $w_k \approx \bar{w}$, per la regolarità dei gradienti $\nabla \ell(w_k; i_k) \approx \nabla \ell(\bar{w}; i_k)$: i due termini stocastici si elidono e resta $g_k \approx \nabla J(\bar{w})$: poca varianza.
- Quando w_k si allontana da $\bar{w}$ la correzione “invecchia”: il rumore non è più eliso e la varianza ricresce $\rightarrow$ si aggiorna l'ancora e si ricampiona l'intero dataset.

Dalla tesi — Appendice D, Schema SVRG



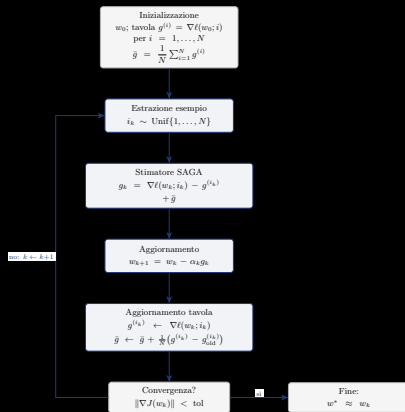
SAGA: la tavola dei gradienti, nessun ricalcolo globale

- Tavola $g^{(i)}$: l'ultimo gradiente calcolato per ogni esempio i ; media $\bar{g} = \frac{1}{N} \sum_i g^{(i)}$ (solo memoria $O(Nm)$, nessuna passata completa periodica).
- Stimatore:

$$g_k = \nabla \ell(w_k; i_k) - g^{(i_k)} + \bar{g}$$

- A ogni iterazione si aggiorna *solo* la voce estratta: $g^{(i_k)} \leftarrow \nabla \ell(w_k; i_k)$ e $\bar{g}$ di conseguenza.
- Stesso schema di riduzione della varianza di SVRG, ma la correzione è sempre “fresca” per l'esempio appena campionato.

Dalla tesi — Appendice D, Schema SAGA (fig. saga)



Limiti di SVRG/SAGA e conseguenze per la tesi

- **SVRG**: il gradiente esatto dell'ancora $\nabla J(\bar{w})$ è ricalcolato ogni m iterazioni con una passata completa sul dataset (costo $O(Nm)$); per N molto grande questi ricalcoli periodici possono dominare il costo complessivo.
- **SAGA**: memoria $O(Nm)$ per la tavola dei gradienti (un gradiente per ogni esempio): improponibile quando $N \cdot m$ è molto grande.
- In entrambi la dimensione del campione è **fissata a priori** (un esempio per iterazione): non è mai adattata alle necessità di accuratezza del momento.
- Le garanzie di convergenza lineare valgono sotto convessità forte.
- **Conclusione**: rendere *dinamica* la dimensione del campione, aumentandola (e ricampionando) quando la stima corrente non è abbastanza accurata: è l'approccio di Byrd, Chin, Nocedal e Wu (2012), al centro della tesi.

Campionamento dinamico: la Condizione di Controllo della Varianza

- Si parte da un batch piccolo: iterazioni economiche; il batch cresce solo quando la stima del gradiente non è abbastanza accurata.
- Criterio (CCV): la varianza campionaria del gradiente deve essere piccola rispetto alla norma del gradiente stimato:

$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|g_k\|_2^2, \quad g_k = \nabla J_{S_k}(w_k), \quad \theta \in (0, 1)$$

- Se la CCV è violata si ricampiona con un batch più grande:

$$n_k^{\text{new}} = \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|g_k\|_2^2} \right\rceil$$

- Profilo a “scalini”; θ piccolo $\rightarrow$ si aumenta presto (batch accurati), θ grande $\rightarrow$ batch piccolo più a lungo.
- Nessuna stima di κ né regola geometrica a priori: la crescita è guidata dai dati.

Dalla tesi — Sez. 5.1 (Byrd, Chin, Nocedal e Wu, 2012)

La CCV garantisce una direzione di discesa

- $-g_k$ è di discesa per J se l'errore relativo della stima è piccolo: $\|e_k\|_2 \leq \theta \|g_k\|_2$ con $e_k = g_k - \nabla J(w_k)$. Infatti, per Cauchy-Schwarz,

$$\nabla J(w_k)^\top g_k = \|g_k\|_2^2 + e_k^\top g_k \geq \|g_k\|_2^2 - \|e_k\|_2 \|g_k\|_2 \geq (1 - \theta) \|g_k\|_2^2 > 0.$$

- In aspettativa l'errore è la varianza dello stimatore:

$$\mathbb{E}[\|e_k\|_2^2] = \mathbb{E}[\|g_k - \mathbb{E}[g_k]\|_2^2] = \|\text{Var}(g_k)\|_1,$$

stimata dalla CCV con $\|\hat{\mathcal{V}}\|_1/n_k$.

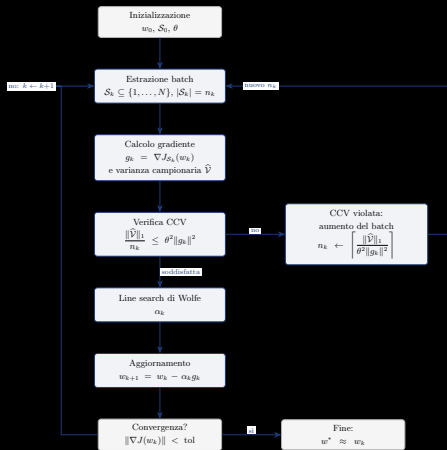
- Sotto la CCV la direzione resta di discesa: è la base per la convergenza dei metodi presentati.

Dalla tesi — Sez. 5.1: condizione di discesa e Lemma di accuratezza

I quattro algoritmi a campionamento dinamico

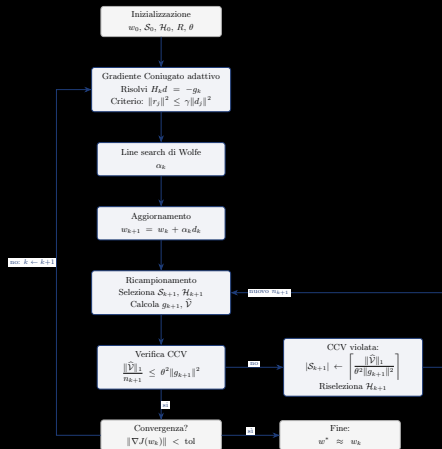
- **Dynamic GD**: gradiente su campione dinamico con CCV e line search di Wolfe (base del metodo).
- **Newton-CG**: Hessiana sottocampionata ($|\mathcal{H}| = R |\mathcal{S}|$) e criterio di arresto adattivo del CG interno, basato sulla varianza dei prodotti Hessiana-vettore: niente iterazioni interne sprecate quando l'Hessiana è limitata dal sottocampionamento.
- **Newton-CG L_1** : gradiente generalizzato, identificazione dell'active set e line search proiettata (soluzioni sparse).
- **BB-CCV**: passo di Barzilai–Borwein con campionamento dinamico (line search di Armijo).
- Tutti implementati in Python (Appendice B) e nell'app web interattiva; per ognuno, lo schema seguente mostra il flusso con la CCV come decisione centrale.

Schema: Dynamic GD (Fig. 5.1)



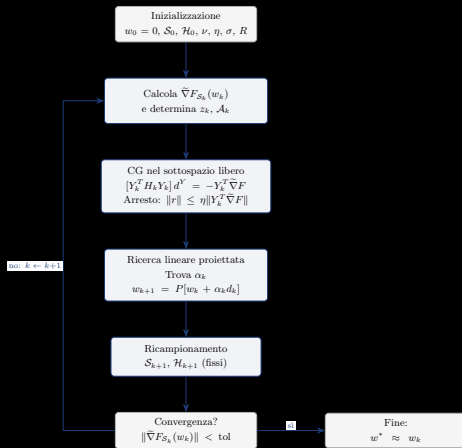
Flusso CCV: se la varianza campionaria è troppo grande rispetto a $\theta^2 \|g_k\|^2$, n_k cresce e si ricampiona; altrimenti line search di Wolfe e aggiornamento.

Schema: Newton-CG (Fig. 5.2)



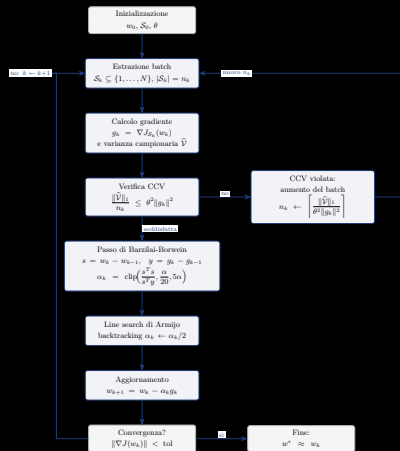
Hessiana sottocampionata con $|\mathcal{H}| = R |\mathcal{S}|$; il CG interno è Hessian-free e si ferma con un criterio adattivo basato sulla varianza dei prodotti Hessiana-vettore.

Schema: Newton-CG L_1 (Fig. 5.5)



Gradiente generalizzato per la L_1 : identificazione dell'active set e line search proiettata mantengono a zero le variabili giuste, producendo soluzioni sparse.

Schema: BB-CCV (Fig. 5.6)



Gradiente dinamico con passo Barzilai–Borwein (line search di Armijo): la CCV regola la dimensione del campione mentre il passo adatta la lunghezza dello spostamento.

Risultati teorici: convergenza e complessità

- Sotto la CCV la direzione di ricerca è di discesa e il metodo converge *linearmente*: sia in forma deterministica sia in aspettativa.
- Complessità totale $\mathcal{O}(mL/(\lambda\varepsilon))$ (con m parametri, L costante di Lipschitz di ∇J , λ convessità forte): competitiva con SGD.
- Su problemi mal condizionati il fattore κ^2 tipico di SGD diventa κ per il campionamento dinamico.
- **Contributo della tesi**: fattore di contrazione migliorato, da $1 - \frac{(1-\theta)\lambda}{2L}$ a $1 - \frac{(1-\theta)\lambda}{L}$, usando la disuguaglianza di Polyak–Łojasiewicz in forma forte.
- Per Newton-CG: arresto adattivo del CG basato sulla varianza dei prodotti Hessiana-vettore, nessuna iterazione interna sprecata.

Dalla tesi — Cap. 5 (convergenza) e Sez. 6.3 (Risultati Numerici)

App web interattiva: esperimenti nel browser

- `visualizzazione.html`: singolo file, Pyodide + Plotly, nessuna installazione o server.
- Codice Python dei quattro algoritmi visibile e modificabile nel browser; preset quadratici + dataset sintetico “centrato” ($\hat{J}_N = J$).
- Percorso 3D/2D su $J(w)$, metriche in tempo reale, pannelli “Analisi” e “Test batch” (per riprodurre le tabelle della tesi).
- Permette di osservare la dinamica del batch (n_k vs k) e l’effetto del parametro θ sulla soglia CCV.
- Implementazione Python standalone equivalente (`simulazione_batch.py`) per la rigenerazione di figure e tabelle della tesi.

Setup e risultati numerici

- Setup: $N = 200$, seed 42, $w_0 = (2, -3)$, $\alpha = 0.1$, $\theta = 0.5$, $n_0 = 5$, 30 iterazioni, tolleranza $\|\nabla J\| < 10^{-6}$.
- Preset: ben condizionato ($\kappa \approx 1.1$), mal condizionato ($\kappa \approx 20$), molto mal condizionato ($\kappa \approx 100$), incrociato.
- Profilo a “scalini” di n_k confermato dagli esperimenti: il batch cresce solo quando la CCV lo richiede.
- BB-CCV raggiunge la precisione macchina ($\approx 1.1 \times 10^{-14}$) su $\kappa \approx 100$; l'analisi teorica (κ al posto di κ^2) è confermata dai dati.

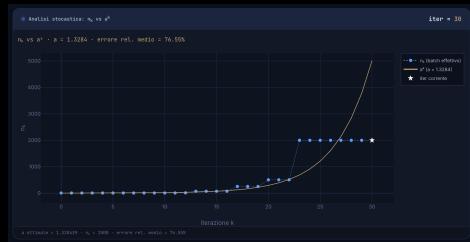
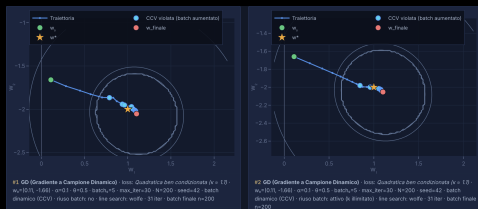


Fig. 5.3 — dimensione del campione n_k vs iterazioni k

Riuso del mini-batch (Sez. 6.5)

- Variante studiata nella tesi: riusare lo stesso campione per M iterazioni consecutive (ricalcolando però il gradiente al punto corrente, non riusando i gradienti vecchi).
- La CCV fa da segnale di “esaurimento” del campione: quando non è più soddisfatta, n_k cresce e si ricampiona.
- $M = \infty$: si tiene il campione finché la CCV vale; il costo per iterazione scende, ma il campione “invecchia”.
- Effetti dipendenti dal metodo e dal condizionamento del problema: quantificati con le tabelle del Cap. 6.



Traiettorie con riuso del batch

Conclusioni

- Quadro completo: dalla varianza del gradiente stocastico (SGD) e dai metodi a varianza ridotta (SVRG/SAGA) si arriva a una regola *dinamica e basata sui dati* per la dimensione del campione.
- Analisi teorica: convergenza lineare (deterministica e in aspettativa) e complessità sotto la CCV.
- Contributi propri: fattore di contrazione migliorato (PL forte), arresto adattivo del CG per Newton-CG.
- Implementazione Python dei quattro algoritmi + app web interattiva per ripetere gli esperimenti.
- Esperimenti sui problemi quadratici sintetici (da $\kappa \approx 1.1$ a $\kappa \approx 100$) coerenti con la teoria; analizzato anche il riuso del mini-batch.

Limiti e lavoro futuro

- Limiti attuali: ipotesi di convessità forte, varianza limitata, esperimenti su dataset sintetici (quadratici, centrati).
- Futuro: assumere direttamente la condizione di Polyak–Łojasiewicz (copre funzioni non convesse).
- Futuro: rapporto di sottocampionamento R dell'Hessiana dinamico (oltre alla sola n_k del gradiente).
- Futuro: validazione su dataset reali e problemi non convessi, confronto sistematico con SVRG/SAGA su larga scala.